

Project Proposal: PebbleDB vs. bbolt

Group Members

- Name: Gokulnath Sourirajan, Email: sourirajan@wisc.edu
- Name: Thilak Raj Murugan, Email: tmurugan2@wisc.edu

Option Selected

We are picking option 1: Evaluating PebbleDB as the storage engine for the kv-store project. Specifically, we compare usage of a different storage engine instead of bbolt in our distributed KV store.

PebbleDB: <https://github.com/cockroachdb/pebble>

Project Idea and Motivation

Our current system uses bbolt for durability in the key-value store and Raft state. We want to investigate whether PebbleDB can improve throughput, write latency, and recovery behavior without increasing implementation complexity too much. This is interesting because PebbleDB is a more modern embedded LSM-tree store, while bbolt is a B+tree-based embedded database. The project is do-able because the persistence layer can be swapped. We then compare the behavior using the same Raft library and client logic.

Metrics We Will Focus On

We will evaluate end-to-end performance and durability by measuring throughput, latency, and restart/recovery behavior under controlled workloads. Below are the specific system metrics we will focus on:

- Throughput under read-heavy and write-heavy workloads
- Average and tail latency for reads and writes
- Recovery time after restart and log replay / state restoration behavior

Workloads and Parameters

We plan to use the existing YCSB-style workloads already used by the project, especially read-heavy mixes like B and write-heavy or balanced mixes like A, E, and F. The most interesting parameters are replication factor, client count, read/write ratio, and key hot-spot concentration. If time permits, we will also compare small-value versus larger-value operations to see whether PebbleDB's performance advantage changes with value size and update frequency.

Build / Run / Testing Environment

We inspected the current repository structure and found that the report artifacts, test cases, and benchmark outputs are already organized in the workspace. The current codebase is Go-based and already supports Raft, demos, fuzzing, and YCSB benchmarking. Our next step is to change the storage engine to PebbleDB and run the YCSB benchmark to observe system metrics.